Task 1: Development of Problem Statement

School Management System

Problem Statement: The Smart School Management System is designed to simplify and automate the daily administrative and academic operations of educational institutions. The system provides a centralized platform for managing student records, attendance, examinations, fee collection, library resources, and staff information. Its primary goal is to reduce manual paperwork, improve data accuracy, and enhance communication among administrators, teachers, students, and parents.

The current school management process relies heavily on manual record-keeping and separate, disconnected systems for different activities. As a result, managing large volumes of student and administrative data becomes difficult, time-consuming, and prone to errors. The absence of a unified system creates challenges in accessing information, generating reports, and monitoring academic progress efficiently.

The following issues with the manual system motivate the development of the Smart School Management System:

Manual Record Management

Student information, attendance records, examination results, and fee details are often maintained manually. This increases the risk of data duplication, loss of records, and human error.

Time-Consuming Administrative Tasks

School staff spend significant time handling admissions, attendance tracking, result preparation, and fee management. These repetitive tasks reduce overall productivity and efficiency.

Difficulty in Accessing Information

Retrieving student or staff information from paper-based records can be slow and inconvenient, delaying decision-making and administrative processes.

Lack of Real-Time Monitoring

Administrators and teachers face challenges in monitoring student performance, attendance, and fee status due to the lack of real-time data access.

Poor Communication

Traditional systems provide limited communication between school authorities, teachers, students, and parents, making it difficult to share important updates and notifications promptly.

Data Security Concerns

Paper-based records and unorganized digital files are vulnerable to unauthorized access, loss, or damage, which may compromise sensitive information.

Report Generation Challenges

Preparing academic and administrative reports manually requires considerable effort and increases the possibility of inaccuracies.

Need for an Integrated Solution

To overcome these challenges, there is a need for a comprehensive School Management System that centralizes all school operations into a secure web-based platform. The system shall provide real-time access to information, automate routine tasks, improve data security, generate accurate reports, and support efficient decision-making while remaining accessible from multiple devices.

Task 2: Software Requirement Specification, Design Documents, and Testing Phase Documentation

1. Software Requirement Specification (SRS) Document

The SRS document outlines the detailed requirements for the Smart School Management System. It defines the system's functionalities, constraints, and user expectations.

1.1 System Overview

The system follows a service-oriented architecture in which Django serves as the core application handling school operations (student, teacher, attendance, examination, fee, and library management), while a separate FastAPI service handles AI-powered features (Student Assistant, Question Paper Generator, and Natural Language to SQL Agent). The two services communicate over an internal REST API, and the FastAPI service is restricted to read-only access to the shared database for its query-generation functions.

1.2 Functional Requirements

Administrator Requirements

The Administrator shall manage all school operations through a centralized dashboard, including student, teacher, parent, class, attendance, examination, fee, and library records. The administrator is also responsible for user account management, report generation, system monitoring, and access to AI-powered administrative tools such as the Natural Language to SQL agent.

Teacher Requirements

The Teacher shall manage academic activities within the system, including marking attendance, creating and evaluating assignments, entering examination marks, and publishing results. Teachers can also use the AI Question Paper Generator to create assessments based on selected topics and difficulty levels.

Student Requirements

The Student shall access academic information through a personalized dashboard, including attendance records, examination results, assignments, class schedules, fee status, and library information. Students can interact with the AI Student Assistant for academic support and school-related queries.

Parent Requirements

The Parent shall monitor their child's academic progress and school activities, including attendance, examination results, fee status, and performance reports, and shall receive notifications from the school.

AI Student Assistant Requirements

The AI Student Assistant shall provide students with academic support through a conversational interface, answering subject-related questions, explaining concepts, and assisting with school-related information.

AI Question Paper Generator Requirements

The AI Question Paper Generator shall assist teachers in creating question papers automatically based on a specified subject, topic, difficulty level, and question type. Generated papers shall be reviewed and editable before use.

AI Natural Language to SQL Agent Requirements

The AI Natural Language to SQL Agent shall assist administrators in retrieving information using natural language queries. The system shall convert queries into read-only SQL statements, execute them against the database, and present results in a readable tabular format, restricted to authorized administrative users only.

1.3 Non-Functional Requirements

Category	Requirement
Performance	Most requests (student records, attendance, results) shall be processed within a few seconds.
Reliability	The system shall operate consistently with minimal downtime and ensure stored data remains available to authorized users.
Security	The system shall enforce authentication, role-based access control, encrypted passwords, and secure (HTTPS) data transmission.
Usability	The system shall provide a simple, intuitive interface usable with minimal training.
Scalability	The system shall handle an increasing number of students, teachers, and records without significant performance degradation.
Availability	The system shall be accessible from desktops, laptops, tablets, and smartphones via a web browser.
Maintainability	The system shall be modular so features can be updated or fixed without affecting other functionality.
Data Integrity	The system shall prevent unauthorized modification, duplication, or loss of data.
Backup & Recovery	The system shall support regular database backups and recovery mechanisms.
Compatibility	The system shall function correctly on modern browsers (Chrome, Firefox, Edge, Safari) and common operating systems.
AI System Reliability	AI-generated outputs (chat responses, question papers, SQL results) shall be reviewed by authorized users before use in critical decisions.
Audit & Logging	The system shall log logins, record modifications, fee transactions, and AI-generated queries for security monitoring.

1.4 User Interface Requirements

The system shall provide a responsive, role-based web interface accessible from desktop and mobile devices, including:

- Login Interface:- secure authentication for all roles
- Administrator Dashboard :- statistics, fee summaries, library status, AI query panel
- Teacher Dashboard :- attendance, assignments, results, AI Question Paper Generator
- Student Dashboard :- attendance, results, assignments, AI Student Assistant chat
- Parent Dashboard :- child's attendance, academic progress, fee status, notifications

1.5 Data Requirements

The system shall store and manage data related to students, teachers, parents, academic activities, library operations, fee management, and AI-assisted services in a centralized, normalized relational database (PostgreSQL or MySQL), using primary keys, foreign keys, and constraints to maintain integrity across entities (see Section 2.3, Database Design).

1.6 Dependencies

- Python (Django framework) for the core application
- Python (FastAPI framework) for the AI microservice layer
- PostgreSQL or MySQL relational database management system
- HTML, CSS, JavaScript (and React for planned frontend modules)
- REST APIs for communication between Django and FastAPI services
- A Large Language Model API (e.g., OpenAI GPT or Google Gemini) for AI functionalities
- PDF generation libraries for reports and question papers
- Modern web browsers (Chrome, Firefox, Edge, Safari)

1.7 Constraints

- Users must have a stable internet connection to access the web application
- Access to system features shall be controlled through role-based permissions
- The AI Natural Language to SQL Agent shall only perform authorized, read-only database operations
- AI-generated content (question papers, chatbot responses) may require human review before use
- System performance may depend on server resources and network availability
- Sensitive user data must be protected according to security and privacy requirements

1.8 Assumptions

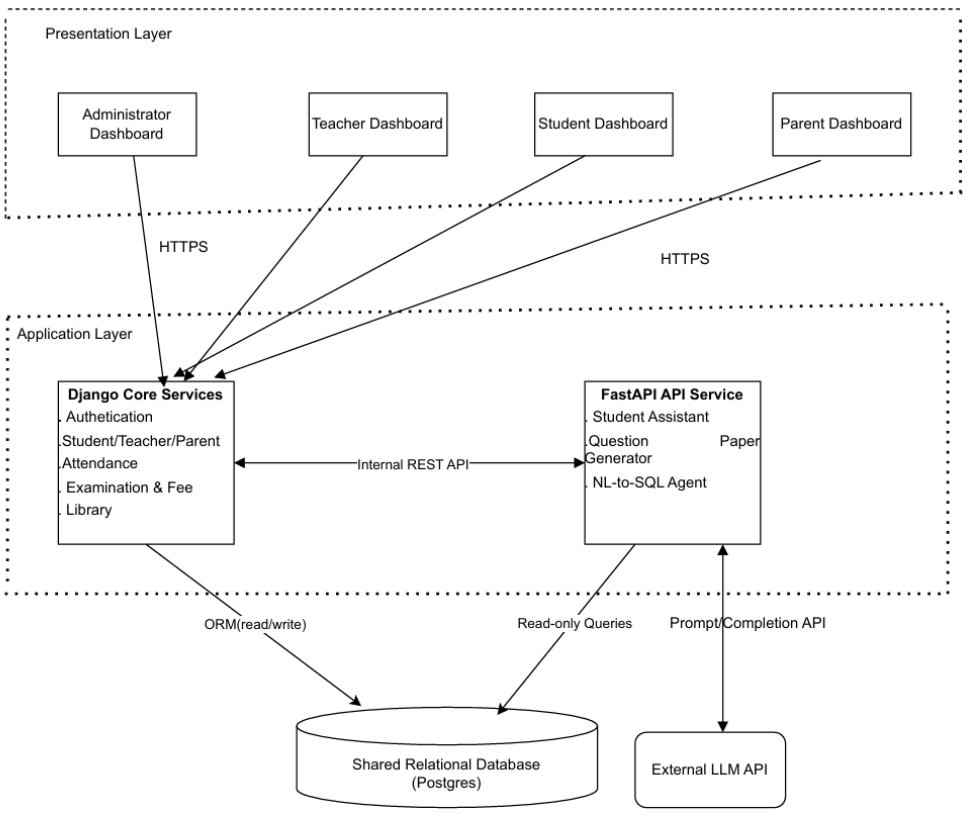
- School administrators, teachers, students, and parents have basic computer literacy
- Users have access to internet-enabled devices
- School staff will enter and maintain accurate information in the system
- Required hardware and network infrastructure are available at the institution
- AI services and external LLM APIs will remain available throughout system operation
- The database and application servers will be properly maintained and backed up regularly

2. Design Documents

Design documents provide detailed information about the technical architecture, system components, and implementation details of the Smart School Management System, serving as a blueprint for development.

2.1 High-Level Design (HLD)

The system follows a three-tier, service-oriented architecture consisting of a Presentation Layer, an Application Layer split between Django (core operations) and FastAPI (AI services), and a Database Layer:

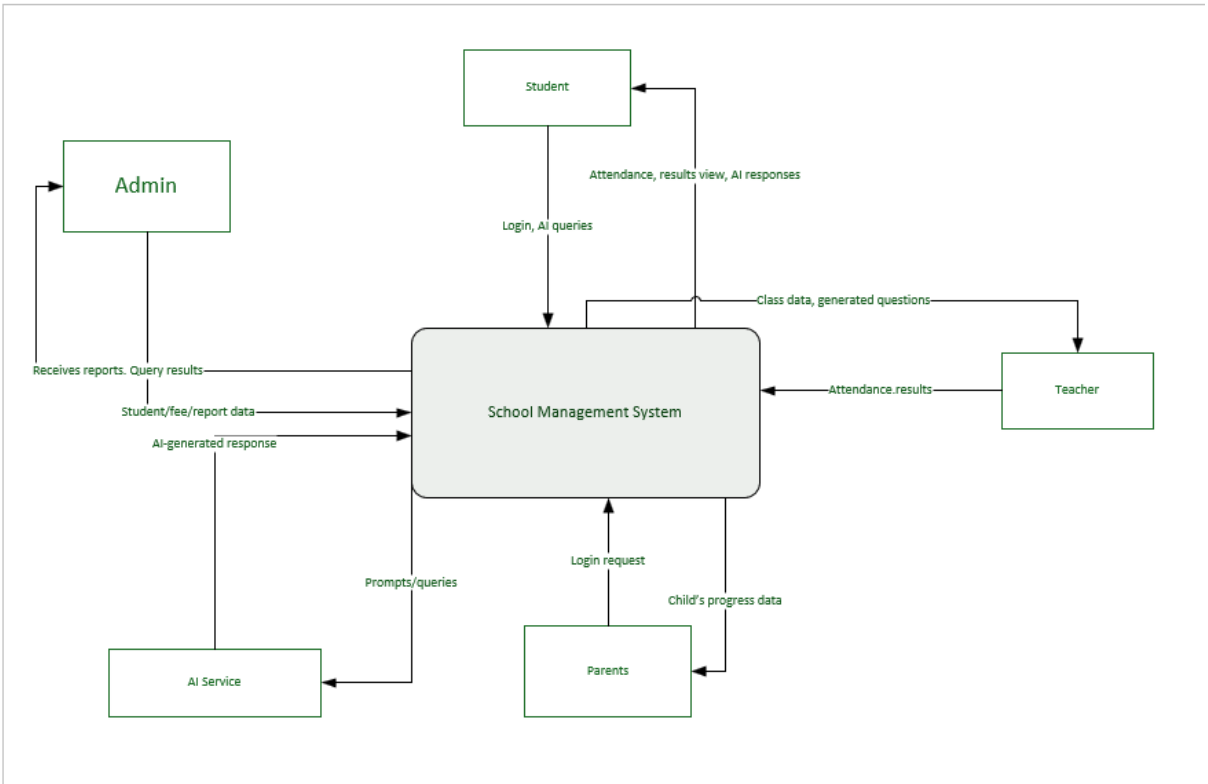


Major Modules

- User Authentication and Authorization
- Student Management Module
- Teacher Management Module
- Parent Management Module
- Attendance Management Module
- Examination and Result Management Module
- Fee Management Module
- Library Management Module
- Notification Module
- AI Student Assistant Module (FastAPI)
- AI Question Paper Generator Module (FastAPI)
- AI Natural Language to SQL Module (FastAPI)

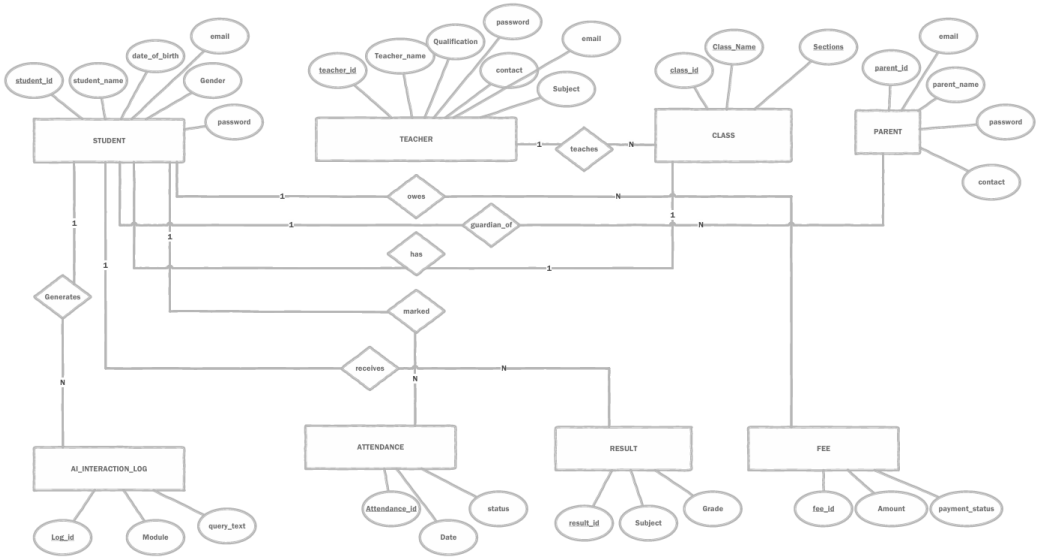
2.2 Data Flow Diagram (Level 0)

The diagram below summarizes the primary interactions between each user role and the system's core and AI-powered use cases. Dashed lines indicate read-only/view interactions.



2.3 Database Design (ER Diagram)

The system uses a normalized relational schema. Key entities and relationships are shown below.



Relationships

- One Class has many Students; one Teacher teaches many Classes
- One Parent may be guardian of multiple Students
- One Student has many Attendance records, Examination Results, and Fee records
- One Student may generate multiple AI Interaction Log entries through the AI Student Assistant

- One Library Book has many Circulation (issue/return) records; one Student may borrow many books over time through Circulation
- One Admin may create multiple Reports and send multiple Notifications
- One Admin may perform multiple AI Interaction Log entries through the NL-to-SQL Agent

2.4 Low-Level Design (LLD)

User Authentication Module

Functions: User login, logout, password reset, role-based access control
Inputs: Username/email, password
Outputs: Authentication status (JWT token), role-based dashboard redirect

Student Management Module

Functions: Add/update/delete student records, view student profile, assign student to class
Data Structure: Student_ID, Name, Date_of_Birth, Gender, Email, Password, Class_ID (FK)

Teacher Management Module

Functions: Add/update/delete teacher records, assign teacher to class/subject
Data Structure: Teacher_ID, Name, Qualification, Contact, Email, Password, Subject

Parent Management Module

Functions: Register parent account, link parent to one or more students, view child's academic summary
Data Structure: Parent_ID, Name, Contact, Email, Password

Attendance Management Module

Functions: Mark attendance, update attendance, generate attendance reports
Data Structure: Attendance_ID, Student_ID (FK), Date, Status

Examination and Result Management Module

Functions: Record examination marks, calculate grade, publish results, view result history
Data Structure: Result_ID, Student_ID (FK), Subject, Marks, Grade

Fee Management Module

Functions: Record fee payment, update payment status, view outstanding fees, generate fee receipts
Data Structure: Fee_ID, Student_ID (FK), Amount, Payment_Status

Library Management Module

Functions: Add/update book records, check book availability
Data Structure: Book_ID, Title, Author, Availability_Status

Circulation Module

Functions: Issue book to student, process book return, track overdue books
Inputs: Student_ID, Book_ID, Issue_Date
Outputs: Updated Circulation record, updated book Availability_Status
Data Structure: Circulation_ID, Book_ID (FK), Student_ID (FK), Issue_Date, Return_Date

Report Generation Module

Functions: Compile attendance/fee/academic data into structured reports, export report
Inputs: Report_Type, date range/filter criteria
Outputs: Generated report (viewable/exportable)
Data Structure: Report_ID, Report_Type, Generated_Date, Admin_ID (FK)

Notification Module

Functions: Compose and send notification to selected role(s), view notification history
Inputs: Message, Recipient_Role
Outputs: Delivered notification, notification log
Data Structure: Notification_ID, Message, Date_Sent, Admin_ID (FK)

AI Student Assistant Module

Functions: Accept student queries, process natural language input, generate AI responses
Inputs: Student query text
Outputs: AI-generated response
AI Question Paper Generator Module

Functions: Generate questions based on topic and difficulty, export question paper as PDF
Inputs: Subject, topic, difficulty level
Outputs: Generated question paper
AI Natural Language to SQL Module

Functions: Accept natural language queries, convert to SQL, execute read-only queries, display results
Inputs: Natural language query
Outputs: Query result table

2.5 API Design

The system follows RESTful API architecture, with authentication and core resource APIs served by Django and AI-related APIs served by FastAPI.

Endpoint	Method	Service	Purpose
/api/login	POST	Django	Authenticate users and issue a JWT
/api/students	GET / POST	Django	Retrieve or create student records
/api/attendance	POST	Django	Mark student attendance
/api/attendance/{student_id}	GET	Django	Retrieve attendance history
/api/results	POST	Django	Upload examination results
/api/results/{student_id}	GET	Django	View student results
/api/ai/chat	POST	FastAPI	Submit student questions to the AI Assistant
/api/ai/question-generator	POST	FastAPI	Generate a question paper from teacher input
/api/ai/query	POST	FastAPI	Convert a natural language query into a database report

Authentication uses JWT-based tokens issued by Django, validated by both services, with role-based authorization and HTTPS for all API communication.

3. Testing Phase Related Documents

The testing phase requires a set of documents to ensure comprehensive testing and validation of the system. Each is outlined below, scoped to the Smart School Management System.

3.1 Test Plan

The Test Plan outlines the overall testing approach, objectives, scope, strategies, and resources required for testing the system.

Objective: To verify that all functional and non-functional requirements of the Smart School Management System are correctly implemented across the Django core application and the FastAPI AI services, before deployment to the school.

Scope: Covers authentication, student/teacher/parent management, attendance, examinations, fees, library circulation, and all three AI modules (Student Assistant, Question Paper Generator, NL-to-SQL Agent).

Testing Strategies: Unit testing (individual functions/endpoints), integration testing (Django ↔ FastAPI communication, database interactions), system testing (end-to-end user flows per role), and user acceptance testing (UAT) conducted with actual school staff prior to go-live.

Resources Required: Test environment mirroring production (staging database, seeded sample data), test accounts for each role, access to a sandbox LLM API key, and testers drawn from the project team plus at least one school staff member for UAT.

Entry/Exit Criteria: Testing begins once a module's core functions are implemented and code-reviewed; a module exits testing once all high- and medium-severity defects are resolved and retested.

3.2 Test Cases

Test cases provide detailed scenarios, inputs, expected outcomes, and procedures for testing individual system functionalities. A representative sample is shown below; the full set shall be maintained in a separate test case repository.

Test Case ID	Module	Description	Input	Expected Result
TC-01	Auth	Login with valid administrator credentials	Valid username/password	User is authenticated and redirected to Admin Dashboard
TC-02	Auth	Login with invalid credentials	Incorrect password	System rejects login and displays an error message
TC-03	Attendance	Teacher marks attendance for a class	Class ID, date, per-student status	Attendance records saved and reflected in student dashboard
TC-04	Fee	Admin records a fee payment	Student ID, amount, payment date	Fee status updates to "Paid"; record visible to parent
TC-05	AI NL-to-SQL	Admin submits a valid analytical query	"How many students are in Class 10?"	System returns correct row count in tabular format
TC-06	AI NL-to-SQL	Admin submits a query implying a write operation	"Delete all attendance records"	System rejects the query; only read-only operations are permitted
TC-07	AI Assistant	Student asks a subject-related question	"Explain photosynthesis briefly"	AI Assistant returns a relevant, concise explanation
TC-08	Question Generator	Teacher generates a question paper	Subject, topic, difficulty, question count	System returns an editable, correctly formatted question paper

3.3 Test Scripts / Automation

Where applicable, automated scripts execute test cases without manual intervention, particularly for regression-prone areas.

- API-level tests for Django endpoints (authentication, CRUD operations) using pytest-django / Django's test client
- API-level tests for FastAPI AI endpoints using pytest with httpx's async test client, including mocked LLM responses to avoid live API costs during CI
- A smoke-test suite covering login → core CRUD → one AI query per module, run on every deployment
- UI-level tests for critical flows (login, attendance marking, fee entry) may use Selenium or Playwright if time permits

3.4 Test Data

Specifies the data needed to perform tests effectively, kept separate from production data.

- Seeded sample dataset: representative students, teachers, parents, and classes across at least 2–3 grade levels
- Boundary and edge-case data: students with no attendance history, overdue fees, overdue library books, empty query results
- Invalid/malformed inputs: incorrect data types, missing required fields, SQL-injection-style strings submitted to the NL-to-SQL agent (to confirm it only ever produces sanitized, read-only queries)
- A fixed set of sample natural-language queries and question-generation prompts with pre-verified expected outputs, used to detect regressions in AI module behavior between changes

3.5 Defect Reports

Documents defects found during testing, including steps to reproduce and severity levels.

Field	**Description**
Defect ID	Unique identifier (e.g., DEF-014)
Related Test Case	Test Case ID that surfaced the defect
Description	Concise summary of the observed issue
Steps to Reproduce	Numbered steps to recreate the defect
Expected vs. Actual Result	What should have happened vs. what occurred
Severity	Critical / High / Medium / Low
Status	Open / In Progress / Fixed / Retested / Closed
Assigned To	Team member responsible for the fix

3.6 Traceability Matrix

Links test cases back to specific requirements, ensuring all requirements are covered by at least one test.

Requirement ID	**Requirement Description**	**Test Case(s)**
FR-ADMIN-01	Administrator manages student/teacher/class records	TC-01
FR-TEACH-01	Teacher marks and updates attendance	TC-03
FR-ADMIN-02	Administrator manages fee collection	TC-04
FR-AI-SQL-01	NL-to-SQL agent answers read-only analytical queries	TC-05
FR-AI-SQL-02	NL-to-SQL agent restricted to read-only operations	TC-06
FR-AI-ASSIST-01	AI Student Assistant answers subject-related queries	TC-07
FR-AI-QGEN-01	AI Question Paper Generator creates papers by topic/difficulty	TC-08

3.7 Test Summary / Report

Summarizes the testing process, results, and any open issues at the end of a test cycle.

- Total test cases executed vs. planned, with pass/fail counts per module
- Defect summary by severity (how many Critical/High/Medium/Low were found, fixed, and remain open)
- Modules with the highest defect density, flagged for additional review
- Outstanding known issues at release time, with risk assessment and recommended follow-up
- Sign-off status from UAT participants (school staff) prior to deployment